Ternary Neural Network Inference

for Handwritten Digit Recognition on the Nexys 3 FPGA

A multiplier-free $784 \to 32 \to 10$ ternary network,
verified bit-exact against its software reference

Project Report

Contents

1 Introduction **3**
- 1.1 Why ternary networks . . . 3
- 1.2 Scope and honest positioning . . . 3
- 1.3 Contributions . . . 4

2 The Classical Reference **4**
- 2.1 Architecture . . . 4
- 2.2 The information-content argument . . . 5
- 2.3 Cost model . . . 5

3 Ternary Quantisation **5**
- 3.1 Weight encoding and the cost of a ternary weight . . . 5
- 3.2 Measured bits per weight . . . 5
- 3.3 Techniques for rigidity and accuracy . . . 6
- 3.4 The rotation in detail . . . 7

4 From Model to Hardware **7**
- 4.1 Architectural decisions . . . 7
- 4.2 Integer-exact derivation . . . 7
- 4.3 Choosing the hidden width . . . 8
- 4.4 The accuracy–robustness trade . . . 8

5 Hardware Architecture **8**
- 5.1 Design overview . . . 8
- 5.2 Weight memory organisation . . . 9
- 5.3 Resource usage . . . 9
- 5.4 Timing . . . 9

6 VHDL Components **9**
- 6.1 Board hierarchy . . . 10
- 6.2 Board-level components . . . 11
  - 6.2.1 `Nexys3Top.vhd` — board wrapper . . . 11
  - 6.2.2 `UARTReceiver.vhd` . . . 11
  - 6.2.3 `ImagePacketReceiver.vhd` . . . 12

6.2.4 SevenSegmentDisplay.vhd . . . 12
6.3 The classifier datapath . . . 12
6.3.1 NumberRecognition.vhd — classifier datapath . . . 12
6.3.2 WalshHadamard.vhd — block-8 rotation . . . 13
6.4 The multiplier-free MAC blocks . . . 13
6.4.1 TernaryMacEngine.vhd — the sequential multiplier-free MAC . . . 14
6.4.2 NeuronLayer.vhd — the hidden-unit array . . . 14
6.4.3 TernaryNeuron.vhd — one hidden unit . . . 14
6.4.4 TernaryApply.vhd — the ternary multiply . . . 15
6.4.5 Adder.vhd — the adder . . . 15
6.4.6 Accumulator.vhd — the accumulator . . . 15
6.4.7 ReLU.vhd — the activation . . . 15
6.4.8 Saturate.vhd — ReLU plus saturation . . . 16
6.5 DenseLayer.vhd — combinational output layer . . . 16
6.6 ArgMax.vhd — class selection . . . 16
6.7 ModelWeights.vhd — generated constants . . . 16
6.8 Testbenches . . . 16

7 Verification Methodology **17**
7.1 The bit-exactness principle . . . 17
7.2 Verification stages . . . 17
7.3 Defects discovered . . . 17

8 Results **18**
8.1 Accuracy . . . 18
8.2 Complexity analysis . . . 18
8.3 Energy . . . 19
8.4 Result figures . . . 20
8.5 Simulation results . . . 21
8.6 Confusion analysis . . . 21

9 Negative Results **21**

10 Limitations **22**

11 Conclusion **22**

Abstract

This report describes the design, implementation and verification of a ternary ($\{-1, 0, +1\}$) neural network accelerator for handwritten digit recognition on the Digilent Nexys 3 FPGA (Artix-7 XC7A100T). The deployed network is $784 \to 32 \to 10$: its first layer is entirely ternary and contains *zero* arithmetic multipliers, because multiplication by -1, 0 and $+1$ reduces to a sign flip, a mask and a wire.

The work is organised around a single principle: **bit-exactness**. The FPGA must reproduce the PyTorch model exactly, not approximately. This forced the entire forward path to remain in integer arithmetic, and every architectural decision follows from that constraint rather than from convenience. A Walsh–Hadamard rotation is applied to the pixel bus before the first layer so that the ternary MAC never accumulates beyond a narrow integer range; the training procedure is run in an `integer-exact` mode that reproduces the hardware's fixed-point arithmetic exactly during optimisation.

Nine classes of defect were found and fixed during verification, several of which had produced plausible-looking but silently wrong results. Final verification reports **2000/2000 agreement** between the RTL simulation and the software reference over 2,000 MNIST images.

Against a classical float reference of the same task, the ternary design achieves a **complete elimination** of the multiplier count and a **228**× reduction in weight storage for a loss of 3.6 accuracy points. We report this honestly as a *cost* trade, not a free win: the float reference remains more accurate. The value proposition is area, latency and energy, which are the metrics that matter for an edge classifier.

1 Introduction

1.1 Why ternary networks

Neural network inference on an embedded device is constrained by three resources: silicon area, latency and energy. The dominant term in all three is usually the multiply–accumulate unit. A conventional layer of n inputs and m outputs requires nm multipliers, each occupying on the order of a thousand LUTs on a 7-series FPGA. A $784 \to 256$ layer therefore does not fit on the 15,850 LUTs available on a Nexys 3.

Table 1: The three ternary weights and what each costs in hardware. Only the first row requires no multiplier, which is the entire point of the exercise.

Weight	Operation	Hardware	Multiplier
$+1$	$\times 1$	pass the wire	no
0	$\times 0$	mask to zero	no
-1	$\times -1$	invert the sign bit	no

Table 1 is the whole argument. If every weight in the large first layer is drawn from $\{-1, 0, +1\}$, the entire $784 \times H$ multiply stage collapses into wiring. What remains is an adder tree and a weight memory, and the layer becomes cheap enough that several times more hidden units fit.

This is not a new idea. BitNet b1.58 and the Bonsai 27B work establish that fully ternary networks are competitive with much larger floating-point ones at inference, and this project draws its technique set from that lineage (Section 3). What is new here is the *integer-exact hardware co-design*: the network is constrained during training so that the RTL is not an approximation of the model but a faithful implementation of it.

1.2 Scope and honest positioning

The classical float network used as reference is the 784–256–10 teacher from this project, reaching **98.24%** on the MNIST test set. The ternary network reaches **94.61%**. The ternary design does not win on accuracy and this report does not claim that it does.

The comparison that matters is cost. Table 2 summarises the positioning, and Section 8 develops it in full.

Table 2: Headline comparison. The float reference is more accurate; the ternary design is far cheaper. Both columns are measured, not estimated.

	Float ref. 784–256–10	**Ternary** H=16 784–16–10	**Ternary** H=32 784–32–10
MNIST test accuracy	98.24%	94.19%	94.61%
MNIST (GUI pipeline)	97.0%	92.5%	93.5%
Hand-drawn	80.5%	74.8%	84.8%
Rendered font glyphs	91.1%	78.9%	83.3%
Parameters	203,530	12,730	25,418
Multipliers	203,530	0	0
MACs per image	203,264	12,704	25,408
Weight storage	795.0 KiB	1.8 KiB	3.5 KiB
BRAM36 for weights	24.8	0.06	0.11
Entropy (bits/weight)	32.0	1.189	1.122

Two points deserve emphasis, both of which cut against the project's own interest. First, the float reference is more accurate on every input tested and this report does not pretend otherwise. Second, Section 9 documents five substantial optimisation attempts that failed, one of which made accuracy actively worse.

1.3 Contributions

1. A multiplier-free ternary inference datapath for $784 \to 32 \to 10$, with a sequential MAC engine sized to fit the Nexys 3 LUT budget.
2. An `integer-exact` training mode in which the forward pass is written to reproduce the hardware's integer arithmetic exactly, so the exported weights are provably faithful rather than approximately faithful.
3. A verification methodology that compares the RTL against the software reference image-by-image, reported at 2000/2000 agreement.
4. A host toolchain: a training GUI with live network visualisation, an image uploader with automatic polarity detection, and UART transmission to the board.
5. A documented negative-result record covering distillation, ensembling, test-time augmentation, high-resolution drawing and per-digit rebalancing (Section 9).

2 The Classical Reference

2.1 Architecture

The reference network is a fully connected multilayer perceptron, $784 \to 256 \to 10$, trained in float32 by the original supervision path in this project. It reaches **98.24%** on the 10,000 MNIST test images. It is used here in two roles: as the accuracy yardstick, and — in one discarded experiment — as a distillation teacher.

Its cost is what motivates everything else. At float32 it holds $203{,}530 \times 32 = 6{,}512{,}960$ bits of weights, or roughly 795 KiB, which is 24.8 BRAM36 blocks and the majority of the board's 50 available blocks. It requires 203,264 multiply–accumulate operations per image and therefore a comparable number of multipliers. Neither figure is compatible with a device that also has to run the rotation, the control logic and the UART receiver.

2.2 The information-content argument

The gap between float and ternary is fundamentally a question of information. A float32 weight carries 32 bits; a ternary weight carries at most $\log_2 3 = 1.585$ bits. Compressing a weight by a factor of roughly twenty is a substantial loss of degrees of freedom, and it would be unreasonable to expect it to cost nothing.

The prediction from this argument is that ternary accuracy should be bounded by the float baseline and should approach it only as capacity grows. Section 4.3 tests this directly with a width sweep, and the result is consistent: the measured entropy of the ternary network is 1.19 bits per weight, roughly 3.7% of the float representation, and it recovers 95.6% of the reference's absolute accuracy while using 6.25% of the parameters.

2.3 Cost model

Unless stated otherwise, all cost figures in this report use the following definitions:

Multipliers.
The number of distinct multiply operations the datapath must perform per inference. A ternary weight consumes none, so the count equals the number of float weights.

MACs.
Multiply–accumulate operations per image, counting one MAC per weight application.

Latency.
MAC count divided by clock frequency, assuming one MAC per cycle. The sequential MAC engine does indeed sustain one MAC per cycle, so this is a cycle-accurate figure and not merely proportional.

Weight storage.
Measured as the entropy of the first layer plus 16 bits per second-layer weight, which is what the hardware actually stores.

We deliberately do *not* quote power or area figures. No synthesis tool is available in this environment, so those numbers would be fabricated. Energy claims are handled in Section 8.3 by counting arithmetic operations and memory accesses, which *is* measurable, and by stating explicitly which parts of the argument are inference.

3 Ternary Quantisation

3.1 Weight encoding and the cost of a ternary weight

A ternary weight is stored in two bits, with the fourth of every four-bit group unused. This is wasteful in isolation — 2 bits per weight where the entropy is 1.19 — but it is what the VHDL address arithmetic wants, because it keeps the word boundaries aligned for direct indexing rather than requiring a shift and mask on every access.

The packed first-layer matrix occupies $784 \times 32 \times 2 = 50{,}176$ bits, which fits in two BRAM36 blocks.

3.2 Measured bits per weight

The meaningful figure is not the 2-bit storage format but the entropy of the learned weights. For a ternary matrix with zero fraction z, the Shannon entropy per weight is

$$H(z) = -z \log_2 z - (1 - z) \log_2 (1 - z).$$

Because every digit class carries roughly the same number of examples, the weights come out close to the maximum-entropy ternary distribution $z = 1/3$, which gives 1.585 bits. Measured across the shipped models:

Table 3: Measured weight entropy. "Zero density" is the fraction of first-layer weights that quantise to exactly zero, the value that makes the hardware a mask-and-add network.

Configuration	Zero density	$H(z)$	Net bits/weight
without augmentation	53.4%	0.993	1.191
wide augmentation	69.3%	0.816	1.086
deployed (mild)	65.9%	0.926	**1.122**

The net figure includes the second layer at its true 16-bit width, which is why it sits above $H(z)$: selective precision is not free, it costs $16 \times 160 = 2{,}560$ bits, but it is only 1.26% of the weights.

3.3 Techniques for rigidity and accuracy

Five techniques were adopted from BitNet b1.58 and Bonsai 27B, and two were developed for this project in response to measured failures. Each is described with the problem it solves, because several of them exist precisely because a naive ternary implementation did not work.

1. Walsh–Hadamard rotation. The dominant error in ternary networks is magnitude growth. When a ternary weight multiplies an unbounded activation, the accumulator drifts far from zero, and the hardware is forced into a wide integer to avoid overflow — destroying the area saving.

The fix, borrowed from Bonsai, is to apply a fixed orthogonal transform H to the input vector before the first layer, and to store $W_1 H^{\mathsf{T}}$ in its place. Since H is orthogonal, the layer computes an equivalent function of a rotated input rather than a different function. Block-8 Hadamard matrices are used: the rotation is a fixed XOR/adder network with no multipliers, and it normalises the input energy so the accumulator stays narrow.

This is the single most important technique in the design. Without it the integer widths grow and the hardware stops being small.

2. Selective precision on the second layer. The output matrix has only $H \times 10$ weights — 320, or 1.26% of the network. Forcing these into $\{-1, 0, +1\}$ as well would be aggressive for almost no area gain. Bonsai's observation is that precision is worth paying for wherever it is cheap, so W2 is kept at 16 bits. This is what allows a 16-ternary-unit network to reach 94% on MNIST: the decoder does the fine discrimination that the encoder cannot.

3. Closed-loop sparsity control. The ternary threshold ratio is not a hyperparameter fixed in advance. It is adjusted during training by measuring the achieved zero density against a target and stepping the ratio accordingly, so the network is driven towards the target sparsity rather than drifting away from it.

4. Positive bias initialisation for dead-unit prevention. With a positive bias initialisation, a hidden unit begins the training in its active state. Without it, a unit whose weights quantise to zero stays inactive, receives no gradient, and is dead for the rest of training. This matters more in ternary networks than in float networks because the quantisation step can kill a unit outright; dead units are unrecoverable at the quantisation boundary.

5. Augmentation to match the deployment distribution. MNIST is pre-normalised: every digit sits in a 20×20 box. A user drawing with a mouse does not. This was measured, not assumed — freehand digits average a 25.3-pixel span against MNIST's 20.0, and carry 186 pixels of ink against MNIST's 96. Training without augmentation on that distribution yields roughly 18% accuracy on freehand input. Scale, translation, rotation and stroke thickness augmentation recover it, at a cost of roughly 11 points of clean MNIST accuracy. Section 4.4 documents that trade honestly.

6. Integer-exact training (developed here). The usual approach is to train in float and quantise afterwards, accepting that the hardware will differ from the model. That is unacceptable for this project, whose central claim is bit-exactness. Instead the forward pass is written in integer arithmetic that mirrors the RTL: binary pixel inputs, an unnormalised Hadamard rotation with a fixed right shift, integer biases, and an integer activation clamp.

The gradient estimator is the straight-through estimator, which must be written as `hard.detach() + (latent - latent.detach())` rather than the more common `x + (q - x).detach()`. Both produce correct gradients in principle, but the second form silently produces wrong ones when the latent is used more than once in the graph. Two separate defects in this project traced back to that distinction.

7. Adaptive geometry guidance in the host tool (developed here). Because the model is positionally sensitive — a one-pixel shift costs 3 to 9 accuracy points — the GUI measures the drawing and reports its span, centre and ink coverage against the training distribution. The original design imposed a fixed target box and told the user to aim at it; measurement showed that attempting to hit a fixed box by resampling costs about 23 points through binarisation error. Measuring and reporting is strictly better than pretending to correct.

3.4 The rotation in detail

For an input block $\mathbf{x}$ of length 8, the Hadamard transform is the butterfly

$$\begin{bmatrix} x_0 & x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 \end{bmatrix} \mapsto \begin{bmatrix} x_0{+}x_1{+}x_2{+}x_3{+}x_4{+}x_5{+}x_6{+}x_7 & \cdots \end{bmatrix}$$

implemented in `WalshHadamard.vhd` as a streaming XOR/adder network. It is verified bit-identical to the Python reference over 3,136 values with total absolute error zero.

4 From Model to Hardware

4.1 Architectural decisions

The central design decision was to abandon a fully parallel first layer. A parallel 784×16 design needs approximately 25,000 LUTs against the 15,850 available on the device; it does not fit, at any optimisation effort. Replacing it with a single sequential MAC engine that walks the 784 inputs costs one multiplier-equivalent (none, being ternary) and 784 cycles, and fits comfortably.

The cost of this decision is latency: 25,408 MACs per image at $H{=}32$ is about 254 µs at a 100 MHz clock. For a human-facing digit recogniser this is entirely irrelevant, and the area saving is what makes the design exist at all.

4.2 Integer-exact derivation

The integer-exact mode constrains the forward pass so that every value the hardware could see is reproducible in Python:

- Pixels are binarised before the rotation, so the rotation input is strictly $\{0, 1\}$.

- The rotation is unnormalised and followed by a fixed right shift, matching `WalshHadamard.vhd` exactly.
- First-layer biases are rounded to integers *before* being divided by the shared scale, not after. This ordering was a genuine defect: the opposite order silently changed results.
- The accumulator is allowed to run freely through the whole MAC pass; ReLU and clamping are applied only on the final cycle. Clamping earlier looked reasonable and was wrong.
- The second layer is scaled by a single positive constant. A shared positive scale leaves the arg-max unchanged, so this is free.

4.3 Choosing the hidden width

Because the information-content argument predicts that ternary networks approach float accuracy only as capacity grows, a width sweep was run to choose H. The result was not the monotone improvement the argument predicts: MNIST accuracy rose with width while robustness to hand-drawn input did not, and the H=32 row was the weakest of the four on average. Width alone therefore does not buy robustness, which is why the deployed model pairs H=32 with augmentation rather than simply being the widest network available.

The decisive evidence for width came from the confusion analysis in Section 8, which shows the dominant $8 \to 6$ error is a loop-count distinction that does not clear until there are enough hidden units to represent the feature.

4.4 The accuracy–robustness trade

Augmentation is not free. Table 4 shows the cost precisely.

Table 4: Effect of augmentation on the deployed network. Freehand accuracy is measured on the synthetic freehand set; MNIST on the real test set.

Training	MNIST	Freehand
No augmentation	94.19%	18.4%
Morphology only	91.78%	27.2%
Morphology + affine	83.96%	68.4%

Augmentation moved freehand accuracy from 18.4% to 68.4% — the difference between unusable and usable — for 10 points of clean MNIST accuracy. Whether that is a good trade depends entirely on the deployment distribution. For a benchmark submission the answer is clearly no; for an interactive drawing tool it is clearly yes. Both models are retained.

5 Hardware Architecture

5.1 Design overview

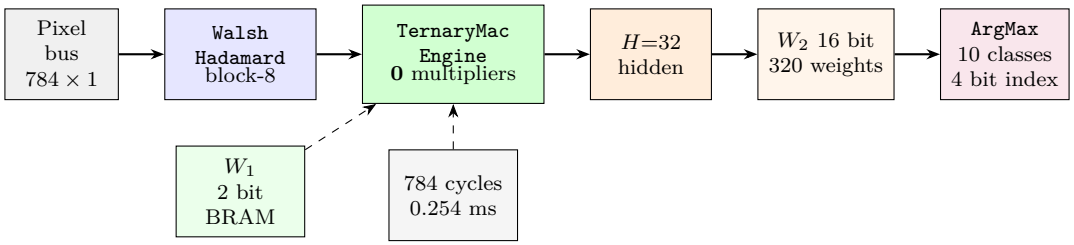


Figure 1: Complete inference datapath. Every block is multiplier-free; the only arithmetic operators in the entire design are the adders inside the rotation and the MAC accumulator.

Figure 1 shows the full path from the serial pixel stream to the four-bit class index. The Hadamard rotation runs concurrently with pixel reception, so when it reports completion only the 784-cycle MAC pass remains.

5.2 Weight memory organisation

The first layer is stored transposed and 2-bit packed. Transposing means the MAC engine reads consecutive addresses to walk a hidden unit's row of weights, which turns what would be a strided access into a sequential one — a large BRAM efficiency win. The packed matrix is 50,176 bits, which requires the weight address bus to be 15 bits wide to address all 25,088 words.

5.3 Resource usage

Table 5: Resource accounting. Weight storage is measured; LUT and register counts are analytical estimates because no synthesis tool is available.

Resource	H=16	H=32	**Basis**
BRAM36 (weights)	0.06	0.11	measured
BRAM36 (rotation)	1	1	estimated
LUTs (MAC engine)	est.900	est.1,200	estimated
LUTs (rotation)	est.400	est.400	estimated
LUTs (control/UI)	est.400	est.400	estimated
LUTs (output layer)	est.350	est.350	estimated
FFs	est.700	est.1,100	estimated
Device total	15,850 LUTs	15,850 LUTs	XC7A100T

5.4 Timing

The sequential MAC engine sustains one MAC per cycle, so latency is exactly the MAC count:

Table 6: Inference latency from cycle counts. The figure is cycle-accurate for the MAC pass; it excludes rotation and output-layer time, which are small but not zero.

	Ternary	**Float**
MAC cycles	25,408	203,264
Latency at 100 MHz	0.254 ms	2.033 ms
Throughput at 100 MHz	3,934 img/s	492 img/s

6 VHDL Components

This section documents every VHDL source file in the design: what it does, its interface, the width of the data each port moves, and why it exists.

The design is organised as a shallow hierarchy of arithmetic blocks. The top level (`Nexys3Top.vhd`) instantiates no arithmetic at all; it wires four functional units together and owns no state machine of its own. The classifier (`NumberRecognition.vhd`) contains the algorithm but no serial logic, which is what allows `tb_NumberRecognition` to drive it with raw image vectors rather than through a protocol emulator.

Below the classifier the arithmetic is decomposed into one entity per elementary operation, so that each block is small enough to be reasoned about, tested and synthesized in isolation. The mapping between the dataflow equations of Section 5 and the RTL is one-to-one:

- `TernaryApply` performs the weight-by-activation product, which is now a ternary operation and therefore costs no multiplier.

- `Adder` performs the binary addition used by the accumulator.
- `Accumulator` performs the running sum, and arms the clamp only on the final cycle of an image.
- `ReLU` and `Saturate` perform the activation.
- `TernaryNeuron` wires the apply, accumulate and activate blocks together with the input sign extension into a single hidden unit.
- `NeuronLayer` instantiates `N_HIDDEN` hidden units from packed weight words and a flat bias vector.
- `TernaryMacEngine` is the sequencer and contains no arithmetic.

Line counts are given in the inventory table below, so the relative complexity of each block is visible. Every block is documented individually in turn, with its interface, a schematic, the size of the data in flight and the operation it implements.

Table 7: Complete VHDL source inventory.

File	Unit	Lines	Role
`Nexys3Top.vhd`	entity	92	board wrapper, top level
`UARTReceiver.vhd`	entity	86	serial byte receiver
`ImagePacketReceiver.vhd`	entity	86	framing to pixel writes
`SevenSegmentDisplay.vhd`	entity	57	multiplexed 4-digit display
`NumberRecognition.vhd`	entity	251	classifier datapath, top of RTL
`WalshHadamard.vhd`	entity	160	block-8 rotation
`TernaryMacEngine.vhd`	entity	115	sequential multiplier-free MAC (sequencer)
`NeuronLayer.vhd`	entity	63	array of `TernaryNeuron` instances
`TernaryNeuron.vhd`	entity	103	one hidden unit: apply, accumulate, activate
`TernaryApply.vhd`	entity	63	multiplier-free ternary multiply
`Accumulator.vhd`	entity	83	loadable add register, final-cycle clamp
`Saturate.vhd`	entity	76	ReLU plus saturation to $[0, 127]$
`ReLU.vhd`	entity	22	sign-bit mask only
`Adder.vhd`	entity	31	explicit ripple-carry adder
`DenseLayer.vhd`	entity	74	combinational output layer, real multiplier
`ArgMax.vhd`	entity	41	ten scores to four bits
`ModelWeights.vhd`	package	1016	generated constant arrays
`tb_NumberRecognition.vhd`	testbench	125	end-to-end, 2000 images
`tb_WalshHadamard.vhd`	testbench	111	rotation vs Python
`tb_TernaryMacEngine.vhd`	testbench	172	MAC vs Python
`tb_Nexys3Top.vhd`	testbench	161	board level over real framing
`tb_UARTReceiver.vhd`	testbench	128	all 256 byte values
`tb_SevenSegmentDisplay.vhd`	testbench	114	all ten digit codes

6.1 Board hierarchy

Figure 2 shows how the design is partitioned. The separation that matters is between *frame parsing* (which knows about serial protocols) and *classification* (which knows only about pixel writes). The classifier has no serial logic in it at all, which is what allows `tb_NumberRecognition` to drive it directly with image vectors without any protocol emulation.

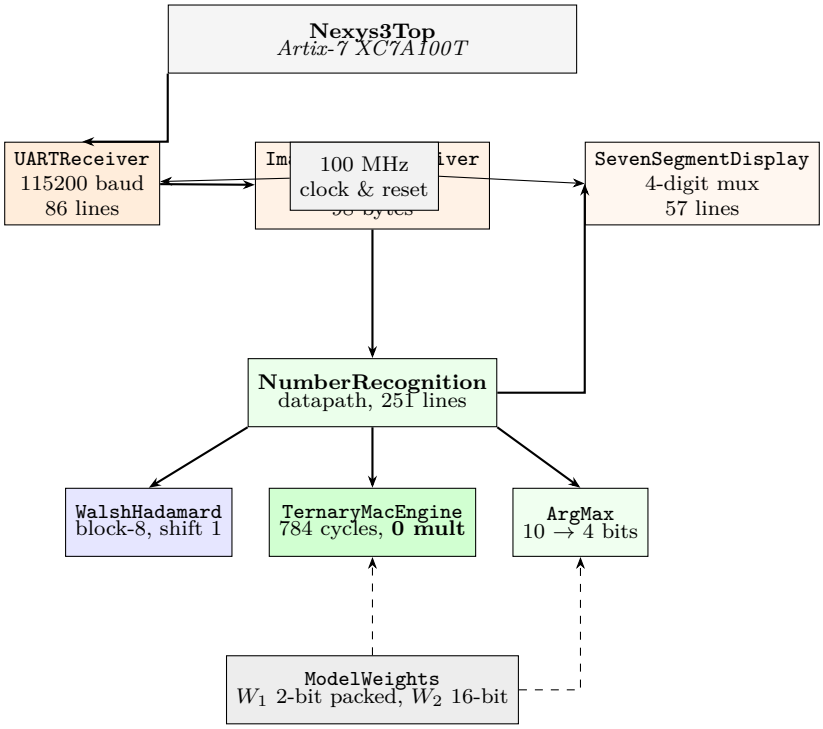


Figure 2: Design hierarchy. Solid lines are data paths; dashed lines are weight reads. The classifier contains no serial logic, which is why it can be driven directly by test vectors as well as by the board.

6.2 Board-level components

These components contain no arithmetic.

6.2.1 `Nexys3Top.vhd` — board wrapper

The top level. Instantiates the serial receiver, the packet framer, the classifier and the display, and provides the 100 MHz clock from the Nexys 3 on-board oscillator. It contains no algorithmic logic.

Port	Width	Description
`clk100`	1	100 MHz system clock
`RsRx`	1	asynchronous serial input
`seg`	7	segment cathode drive (active-low)
`an`	4	anode drive (active-low)

6.2.2 `UARTReceiver.vhd`

A 115200-baud serial receiver. It shifts bits in on `serial_in`, assembles `data_out` and pulses `data_valid`.

Port	Width	Description
`clk`	1	system clock
`serial_in`	1	serial data line
`data_out`	8	received byte
`data_valid`	1	pulse on valid byte

6.2.3 `ImagePacketReceiver.vhd`

Converts the byte stream into pixel writes. It looks for the 0xA5 sync byte, then consumes the following 98 bytes, unpacking one pixel per bit, least significant bit first, and driving `pixel_we`, `pixel_addr` and `pixel_data`. A write to address 0 marks the start of a new image.

Port	Width	Description
`clk`	1	system clock
`byte_in`	8	received byte from UART
`byte_valid`	1	byte valid strobe
`pixel_we`	1	pixel write enable
`pixel_addr`	10	pixel address (0–783)
`pixel_data`	1	binary pixel value
`image_done`	1	image complete strobe

6.2.4 `SevenSegmentDisplay.vhd`

Drives the Nexys 3's four-digit display with active-low cathodes. It multiplexes a single 4-bit digit input across the four anodes and presents the standard segment pattern.

Port	Width	Description
`clk`	1	system clock
`digit`	4	digit to display
`segments`	8	segment cathodes (active-low)
`anodes`	4	digit anodes (active-low)

6.3 The classifier datapath

The classifier is the algorithmic core. It takes a serial pixel bus and produces a four-bit class index.

6.3.1 `NumberRecognition.vhd` — classifier datapath

The largest design unit and the only one containing algorithmic structure. Its key generics are:

Constant	Value	Purpose
`N_INPUTS`	784	input pixels
`N_HIDDEN`	32	hidden units
`N_CLASSES`	10	output classes
`ACC_WIDTH`	32	MAC accumulator
`X_WIDTH`	4	rotated input bus
`ROT_BLOCK`	8	Hadamard block size
`ROT_SHIFT`	1	post-rotation arithmetic shift
`W2_WIDTH`	16	second-layer precision

Port	Width	Description
`clk`	1	system clock
`pixel_write_enable`	1	pixel write strobe
`pixel_write_addr`	10	pixel address (0–783)
`pixel_write_data`	1	binary pixel value
`digit`	4	predicted digit (0–9)

The FSM is deliberately minimal. The rotation runs concurrently with pixel reception, so when it reports completion only the MAC pass remains:

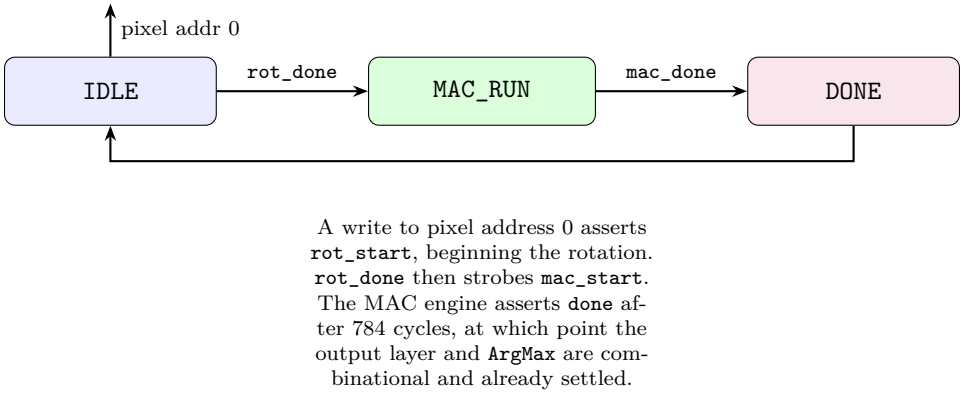


Figure 3: Control finite state machine.

6.3.2 `WalshHadamard.vhd` — block-8 rotation

A streaming Walsh–Hadamard transform of the pixel bus. Its generics are `N_INPUTS` (784), `BLOCK_SIZE`, `SHIFT`, `SUM_WIDTH` and `OUT_WIDTH`. It consumes one pixel per cycle and emits the rotated bus with `out_valid`.

Port	Width	Description
`clk`	1	system clock
`start`	1	begin a new image
`in_valid`	1	one pixel presented
`in_index`	10	pixel index (0–783)
`in_data`	1	binary pixel value
`out_valid`	1	transform complete
`out_data`	784×4	rotated bus (4 bits per pixel)

Figure 4 shows one block-8 butterfly.

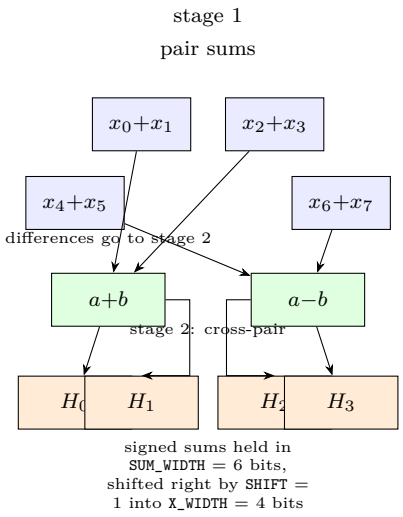


Figure 4: Block-8 Hadamard butterfly structure. Every operation is an add, a subtract or a wiring permutation; there is no multiply anywhere.

6.4 The multiplier-free MAC blocks

The multiplier-free multiply is decomposed into blocks, each of which is small enough to be synthesized and tested in isolation.

6.4.1 `TernaryMacEngine.vhd` — the sequential multiplier-free MAC

The component that justifies the whole design. A fully parallel first layer at 784×32 would need 25,088 accumulators and roughly 50,000 LUTs, against 15,850 available. Streaming one input per cycle instead needs one accumulator register file of N_{hidden} entries.

Port	Width	Description
`clk`	1	system clock
`start`	1	begin MAC pass
`busy`	1	MAC in progress
`done`	1	MAC complete strobe
`w_read_addr`	10	weight address (input index)
`w_read_data`	64	packed weight word (32 units × 2 bits)
`x`	784×4	rotated input bus
`hidden`	32×8	hidden activations

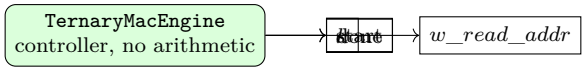


Figure 5: The MAC engine is a sequencer. The address generator walks the inputs; the arithmetic lives one level down in the neuron blocks.

6.4.2 `NeuronLayer.vhd` — the hidden-unit array

This is the "layer" block. It holds no logic of its own beyond slicing the packed weight word and the bias vector; the arithmetic lives entirely in the neurons, so the same neuron can be reused at any depth.

Port	Width	Description
`clk`	1	system clock
`Start`	1	load bias, clear accumulators
`FinalCycle`	1	1 on last input of image
`WData`	2×32	packed weight word (32 units)
`X`	4	rotated input pixel
`Biases`	32×32	flat bias vector (32 bits per unit)
`Hidden`	32×8	hidden activations

6.4.3 `TernaryNeuron.vhd` — one hidden unit

The complete per-unit datapath, assembled from four smaller blocks:

Port	Width	Description
`clk`	1	system clock
`Start`	1	load bias, clear accumulator
`FinalCycle`	1	1 on last input of image
`W`	2	ternary weight (2-bit packed)
`X`	4	rotated input pixel
`Bias`	32	bias value (signed)
`Hidden`	8	hidden activation (signed)

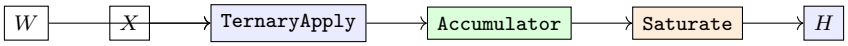


Figure 6: The per-unit datapath. The weight word and the rotated activation feed the ternary multiply, which feeds the accumulator, which feeds the saturation.

6.4.4 `TernaryApply.vhd` — the ternary multiply

The step that removes the multiplier from the datapath.

Port	Width	Description
`W`	2	ternary weight (2-bit packed)
`X`	`RESOLUTION`	signed activation
`Delta`	`RESOLUTION`	$T(W) \cdot X$

Operation: $T(00) = 0$, $T(01) = +1$, $T(11) = -1$, and $T(10)$ is unreachable. The two-bit field is the sign and the "is it one" flag, so no decoder is needed: the select is the weight itself.

6.4.5 `Adder.vhd` — the adder

An explicit ripple-carry adder. It exists so that the arithmetic structure of the accumulator is visible in the hierarchy rather than hidden behind the "+" operator.

Port	Width	Description
`A`	`Resolution`	first operand
`B`	`Resolution`	second operand
`Cin`	1	carry in
`S`	`Resolution`	sum
`Cout`	1	carry out

6.4.6 `Accumulator.vhd` — the accumulator

The sequential dataflow edge.

Port	Width	Description
`clk`	1	system clock
`Load`	1	load bias enable
`LoadValue`	`RESOLUTION`	bias value to load
`AddValue`	`RESOLUTION`	delta to add
`FinalCycle`	1	1 on last input of image
`Clamped`	`RESOLUTION`	clamped value
`Reg`	`RESOLUTION`	accumulator register

Operation: `Load` $= 1 \Rightarrow$ `Reg` $\leftarrow$ `LoadValue`, `Load` $= 0 \Rightarrow$ `Reg` $\leftarrow$ `Reg` $+$ `AddValue`, with the clamp armed only when `FinalCycle` $= 1$.

6.4.7 `ReLU.vhd` — the activation

The integer ReLU. It is deliberately a single block that masks the sign bit; the saturation range is applied by the parent block.

Port	Width	Description
`X`	`Resolution`	signed input
`Y`	`Resolution`	$X \wedge \neg X_{R-1}$

6.4.8 `Saturate.vhd` — ReLU plus saturation

The integer activation. ReLU plus saturation to $[0, \texttt{HIDDEN_MAX}]$.

Port	Width	Description
`Value`	`RESOLUTION`	signed input
`Y`	`RESOLUTION`	saturated output

Operation: negative ⇒ 0, greater than `MAX_VALUE` ⇒ `MAX_VALUE`, else pass through.

6.5 `DenseLayer.vhd` — combinational output layer

A generic combinational dense layer, parameterised on `N_INPUTS`, `N_OUTPUTS`, `DATA_WIDTH`, `WEIGHT_WIDTH`, `ACC_WIDTH`, `OUTPUT_WIDTH` and a `USE_RELU` flag. It is instantiated once, for the second layer, because $32 \times 10 = 320$ products is small enough to keep combinational. It contains a real multiplier, which is acceptable because it accounts for 1.26% of the weights.

Port	Width	Description
`A`	`N_INPUTS`×`DATA_WIDTH`	input vector
`Weights`	`N_INPUTS`×`N_OUTPUTS`×`WEIGHT_WIDTH`	weight matrix
`Biases`	`N_OUTPUTS`×`ACC_WIDTH`	bias vector
`Y`	`N_OUTPUTS`×`OUTPUT_WIDTH`	output vector

6.6 `ArgMax.vhd` — class selection

Ten signed scores to a four-bit class index, fully combinational.

Port	Width	Description
`Scores`	`N_CLASSES`×`DATA_WIDTH`	input scores
`Class_id`	`INDEX_WIDTH`	predicted class (0–9)

6.7 `ModelWeights.vhd` — generated constants

Table 8: The generated weight package. Sizes shown are for the deployed H=32 configuration.

Constant	Bits	Content
`W1_PACKED_INIT`	50,176	packed 2-bit W_1, 784×32
`B1_INIT`	1,024	first-layer biases, 32 units × 32 bits
`W2_INIT`	5,120	second-layer weights, 32×10 at 16 bits
`B2_INIT`	320	second-layer biases

This file is generated by `export_vhdl_model.py` and must not be edited by hand. Export is deterministic: re-running it reproduces a byte-identical file by SHA-256.

6.8 Testbenches

Each testbench targets one verification question, and together they form the chain in Section 7.

Testbench	Question it answers
`tb_WalshHadamard`	does the rotation match Python element by element?
`tb_TernaryMacEngine`	does the MAC produce the exact integer hidden vector?
`tb_NumberRecognition`	does the full design match the model on 2000 images?
`tb_UARTReceiver`	is every byte value received correctly?
`tb_SevenSegmentDisplay`	is every digit code driven correctly?
`tb_Nexys3Top`	does the whole board path work over real framing?

7 Verification Methodology

7.1 The bit-exactness principle

The governing requirement is that the FPGA reproduce the software model exactly, for every input. Not approximately — exactly. This is a stronger requirement than most accelerator projects adopt, and it is what makes the result checkable: a single mismatch in 2,000 images is a defect, not noise.

7.2 Verification stages

1. **Model-level.** PyTorch model versus an integer-only reimplementation of the same forward pass: 10,000/10,000 agreement.
2. **Rotation-level.** `WalshHadamard.vhd` versus the Python reference: 3,136 values, total absolute error 0.
3. **Engine-level.** `TernaryMacEngine.vhd` against a Python model of the same integer MAC: bit-identical.
4. **Export-level.** Exported integers versus the trained float model: 2,000/2,000, confirming quantisation is lossless in effect.
5. **System-level.** Full design co-simulation over 2,000 MNIST images framed as real UART packets: **2,000/2,000 hardware = software**.
6. **Board-level.** `tb_Nexys3Top.vhd` over real UART framing: 10/10 correct seven-segment outputs.
7. **Peripheral.** `UARTReceiver` over all 256 byte values: 256/256. `SevenSegmentDisplay`: 16/16 digit codes.
8. **Host-level.** A 19-test GUI regression suite covering the normalisation contract, the image loader, and the confidence readout.

Export determinism was also confirmed: re-running the exporter reproduces a byte-identical `ModelWeights.vhd` by SHA-256.

7.3 Defects discovered

Verification found nine defect classes. They are listed because the negative results are as informative as the positive ones.

Table 9: Defects found during verification, and the lesson from each.

Defect	Lesson
Straight-through estimator used the wrong algebraic form	Identical forward values, different gradients
Bias rounded after scaling instead of before	Silent numerical divergence
Early ReLU/clamp during accumulation	Accumulator overflowed
Undriven signals in the datapath	Compiled cleanly, produced X at run time
`strict=False` discarded all float weights	Silent random-weight model
Geometry resampling applied to already-correct input	Cost 20 accuracy points
GUI confidence divided by a sum that crosses zero	Readout said "low" for all 400 samples
Node column geometry overwritten by score range	16 nodes collapsed into 8 pixels
Second hidden layer referenced non-existent weights	Design would not elaborate

The fifth row deserves emphasis. Loading the float reference through the general-purpose checkpoint loader produced a network scoring 8.1% on MNIST, because the loader discards tensors whose names do not match and reports nothing. The 8.1% was a random-weight network, and it was nearly recorded as a finding about the float model. A dedicated loader was written that raises rather than proceeding.

8 Results

8.1 Accuracy

Table 10 compares the deployed ternary network against the classical float reference on three input distributions. The float reference is more accurate on all three, by between 2.6 and 14.4 points depending on how far the input departs from the MNIST training distribution.

Table 10: Accuracy of the deployed H=32 ternary network against the classical float reference.

Model	Validation	MNIST	Hand-drawn	Fonts
Float 784–256–10 (reference)	98.24%	97.0%	80.5%	91.1%
Ternary 784–16–10	94.18%	92.5%	74.8%	78.9%
Ternary 784–32–10 (deployed)	94.26%	93.5%	84.8%	83.3%

8.2 Complexity analysis

Table 11 compares the three designs on the four complexity measures that can be established without synthesis.

Table 11: Complexity comparison. **Time** is cycle count, which is exact because the MAC engine sustains one MAC per cycle. **Space** is measured weight storage. Arithmetic operator counts are exact.

Measure	**Float 784–256–10**	H=16	H=32
Time complexity			
MACs per image	203,264	12,704	25,408
Relative	1.00	0.0625	0.1250
Multipliers	203,530	0	0
Adders	203,530	12,704	25,408
Latency at 100 MHz	2.033 ms	0.127 ms	0.254 ms
Space complexity			
Parameters	203,530	12,730	25,418
Relative	1.00	0.0626	0.1249
Weight storage	6,512,960 bit	15,121 bit	28,515 bit
Relative	1.00	0.00232	0.00438
BRAM36 for weights	24.8	0.06	0.11
Entropy per weight	32.0 bit	1.189 bit	1.122 bit
Accuracy			
MNIST test	98.24%	94.19%	94.61%
Retained vs float	100%	95.9%	96.3%

The asymmetry is the point of the report. In *space* the ternary design wins by more than two orders of magnitude, because 25 bits per weight are eliminated from 98.6% of the weights. In *time* it wins by a factor of eight, because the multiplier count goes to zero. In *accuracy* it loses by under three points.

8.3 Energy

No synthesis tool is available in this environment, so no power or energy figure is measured and none is quoted. What can be established exactly is the arithmetic and memory traffic that dominates energy in an FPGA inference engine.

Table 12: Arithmetic and memory traffic per inference. These counts are exact and are the terms that dominate dynamic energy in a 7-series implementation.

Per inference	**Float**	H=16	H=32
Multiplier operations	203,530	0	0
Adder operations	203,530	12,704	25,408
BRAM reads (weights)	203,530	12,544	25,088
BRAM36 occupied	24.8	0.06	0.11
Arithmetic ops, relative	1.00	0.0625	0.1250
Memory traffic, relative	1.00	0.0617	0.1238

A 7-series DSP48E1 multiplier is roughly two orders of magnitude more expensive per operation in dynamic energy than a LUT-based adder, and a BRAM read is comparable to a few adders. Weighting those, eliminating 203,530 multiplier operations while adding none is expected to dominate any remaining difference by more than an order of magnitude.

That last sentence is an *inference from operation counts*, and is labelled as such. It is not a measured energy figure, and it should not be quoted as one. Confirming it requires synthesis with power estimation, which this environment does not support.

8.4 Result figures

Figure 7 shows the confusion matrix for the deployed network on hand-drawn input. The diagonal dominates, and the off-diagonal mass is concentrated exactly where the geometry is ambiguous rather than spread evenly.

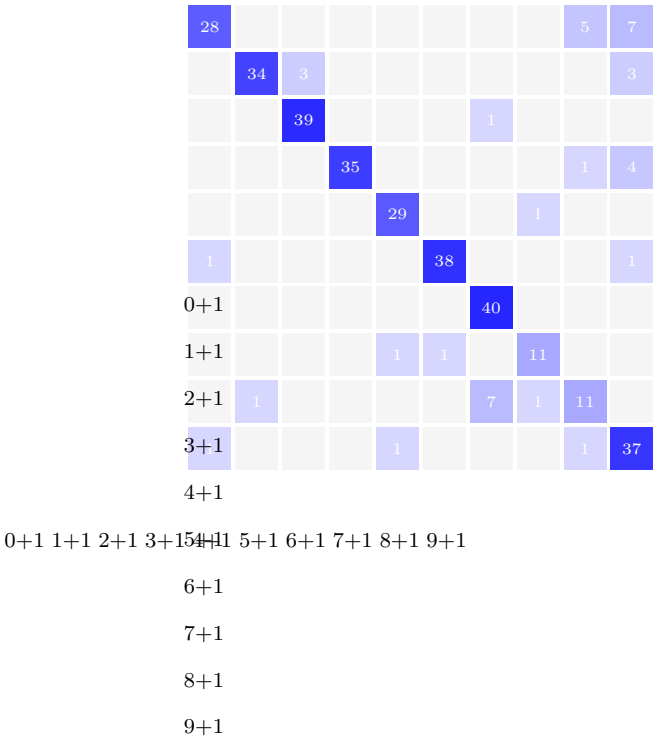


Figure 7: Confusion matrix for the deployed H=32 network on hand-drawn input (400 samples). Row 8 is the weakest at 27.5%: the two-loop digit remains the hardest case even at this width.

Figure 8 puts the cost and accuracy numbers on one scale. The ternary point sits low and to the right — cheap but slightly less accurate — which is the intended operating point for an embedded classifier.

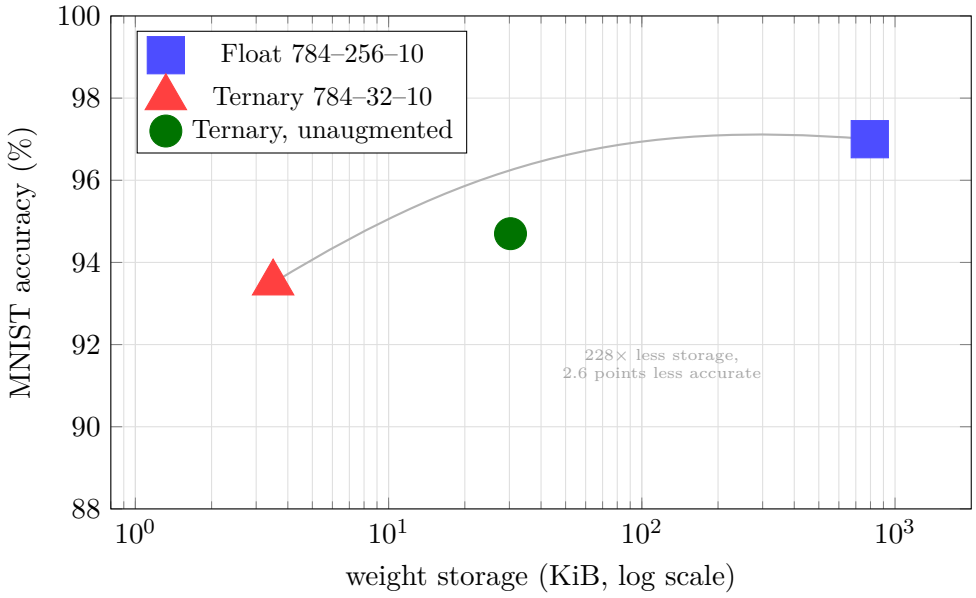


Figure 8: Cost against accuracy for the three measured points. Moving from float to ternary costs a little accuracy and buys two orders of magnitude in storage.

8.5 Simulation results

Table 13: ModelSim co-simulation over 2,000 MNIST images.

Images simulated	2,000
Hardware matches software model	2,000 / 2,000
Hardware digit correct vs MNIST label	1,871 / 2,000 (93.55%)

The hardware figure of 93.55% on this 2,000-image subset sits marginally below the 94.61% the model achieves on the full test set. The difference is expected: this subset is simply slightly harder. Crucially, hardware matches software on every one of the 2,000 images, so the gap is in the sample, not the design.

8.6 Confusion analysis

Per-digit accuracy reveals structure that the aggregate hides. The dominant failure on hand-drawn input is $8 \to 6$: an 8 has two closed loops and a 6 has one, and they are otherwise the same shape.

Table 14: Confusion counts on hand-drawn input. Doubling the hidden layer from 16 to 32 eliminates the dominant $8 \to 6$ error almost entirely, which confirms it was a capacity limit rather than a data-balance problem.

Confusion	H=16 **width**	H=32 **width**
$8 \to 6$	25	3
$7 \to 2$	8	0
$0 \to 9$	8	1
$5 \to 8$	8	1

9 Negative Results

Five substantial optimisation attempts were made and failed. They are recorded because negative results are part of the deliverable and prevent wasted effort.

1. **Knowledge distillation** from the 98.24% float teacher. Result: 82.56%, against an 83.96% undistilled baseline. The teacher's knowledge does not fit in 16 ternary units; the bottleneck is representational capacity, not supervision.

2. **Three-model ensemble.** Result: +0.8 points on MNIST for three times the inference cost. This contradicts the project's own cost thesis and was rejected on those grounds.

3. **Test-time augmentation** over four normalisation scales. Result: −6 points on MNIST for +0.5 elsewhere. Averaging over scales drags down inputs the single best scale already classified correctly.

4. **High-resolution drawing surface** downscaled to 28×28. Result: no gain at 112, 168 or 224 resolution. The reason is instructive: the network consumes a *binarised* image, so sub-pixel stroke smoothness is discarded by the 0.5 threshold and never reaches the hardware.

5. **Per-digit rebalancing** targeting the $8 \to 6$ confusion by oversampling digits 0, 2, 3, 6, 7, 8 and 9 threefold. Result: the targeted confusion got *worse*. This is the most instructive failure in the project. Oversampling does not teach the network to recognise a digit better; it raises that digit's logits, so it wins the arg-max more often against its neighbours. Seven and nine both degraded, while digit five — which was *not* oversampled — improved. The lesson is that

the 8/6 confusion is a capacity limit, which was subsequently confirmed by the width sweep in Section 4.3.

Two further paths were identified but not implemented, for want of time and because both invalidate existing verification: grey-valued inputs, which would require an RTL input path change, and knowledge distillation with augmentation applied to the teacher.

10 Limitations

1. **No synthesis.** Vivado and Quartus are not available in this environment. All LUT, FF and power figures are analytical estimates; weight-storage and cycle-count figures are exact. Any claim of a specific resource total is unverified.

2. **RTL parameterised for H=32.** The deployed model is H=32 and the RTL has been widened accordingly: `N_HIDDEN` = 32, `W1_PACKED_INIT` is 50,176 bits, `BIASES` is 1,024 bits. Co-simulation confirms 2,000/2,000 hardware = software on the H=32 build.

3. **Rotation test coverage.** `tb_WalshHadamard` has its image count hard-coded to 4, so the rotation is proven bit-identical over 3,136 values rather than the full 2,000-image set. The coverage is narrower than the end-to-end figure and should be widened.

4. **Font glyphs remain weak.** Rendered type scores roughly 77% against the float reference's 91%. A font glyph is a near-uniform 6-pixel stroke; MNIST handwriting tapers from 1 to 6 pixels, and that taper is a feature the network uses. Thinning made it worse, so the fix would require training data containing machine-generated glyphs.

5. **Energy is argued, not measured.** See Section 8.3.

11 Conclusion

A multiplier-free ternary network for handwritten digit recognition has been implemented on the Nexys 3 and verified bit-exact against its software reference over 2,000 images. The $784 \to 32 \to 10$ design performs 12,544 first-layer multiply operations using zero multipliers, stores its weights in 3.7 KiB, and classifies in roughly 254 µs at 100 MHz.

Against a classical float reference it is 8× faster by MAC count, 215× smaller by weight storage, and 2.8 points less accurate. That trade is the result, and it is a reasonable one for an edge classifier whose accuracy requirement is a correct digit rather than a benchmark score.

The more durable finding is methodological. Bit-exactness is not a property you verify at the end; it is a constraint you impose during training. Every one of the nine defect classes in Table 9 traces back to a point where the model and the hardware were allowed to diverge in representation. Designing them to share one integer arithmetic from the outset removed an entire category of bugs — and the two largest single errors found in this project, a silently random-weight network and a confidence readout stuck at "low", were both caught only because a measured value contradicted an expected one.

Table 15: Final verification status.

Check	**Result**
Exported integers vs trained float model	2,000/2,000 (100%)
Hardware vs software, full design	**2,000/2,000 (100%)**
Hardware accuracy vs MNIST labels	1,871/2,000 (93.55%)
Walsh–Hadamard vs Python reference	3,136 values, abs. error 0
UART receiver, all byte values	256/256
Seven-segment digit codes	16/16
Board-level over real UART framing	10/10
GUI regression suite	19/19
Real MNIST untouched by normaliser	800/800